

INSTITUTO FEDERAL DE
EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
SUDESTE DE MINAS GERAIS
Campus Juiz de Fora



Programação I

Professor Roberto

roberto.ferreira@ifsudestemg.edu.br

Aula 03 – Sistema Agenda
API

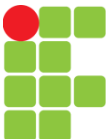
Sistema Agenda

- API
 - Application Programming Interface (Interface de Programação de Aplicação)
 - Contrato de serviço entre duas aplicações
 - Define como será a comunicação
 - Formato da solicitação e da resposta



Sistema Agenda

- API
 - Vou disponibilizar uma API REST
 - REST significa Transferência Representacional de Estado. REST define um conjunto de funções como GET, PUT, DELETE e assim por diante, que os clientes podem usar para acessar os dados do servidor. Clientes e servidores trocam dados usando HTTP.
 - As solicitações do cliente ao servidor são semelhantes aos URLs que você digita no navegador para visitar um site. A resposta do servidor corresponde a dados simples, sem a renderização gráfica típica de uma página da Web.



Sistema Agenda

- Como vamos acessar a API
 - A partir de um endpoint (endereço)
 - Vamos usar requisições HTTP para solicitar recursos
 - Vamos receber um código de resposta para cada solicitação (Códigos de status de respostas HTTP) e dados no formato JSON
 - Vamos utilizar programação assíncrona



Sistema Agenda

- Métodos de requisição HTTP
 - O protocolo HTTP define um conjunto de métodos de requisição para indicar a ação a ser executada
 - HTTP Verbs (Verbos HTTP)
 - <https://developer.mozilla.org/pt-BR/docs/Web/HTTP/Methods>



Sistema Agenda

- Métodos de requisição HTTP
 - Principais
 - GET
 - Utilizado para obter dados e informações do servidor
 - POST
 - Utilizado para enviar ou criar dados no servidor
 - PUT
 - Utilizado para atualizar dados completamente
 - PATCH
 - Utilizado par atualizar dados parcialmente
 - DELETE
 - Utilizado para excluir dados do servidor



Sistema Agenda

- Código de status de respostas HTTP
 - Respostas Informativas
 - 100 – 199
 - Respostas bem-sucedidas
 - 200 – 299
 - Mensagens de redirecionamento
 - 300 – 399
 - Respostas de erro do cliente
 - 400 – 499
 - Respostas de erro do servidor
 - 500 – 599
- <https://developer.mozilla.org/pt-BR/docs/Web/HTTP/Status>



Sistema Agenda

- Código de status de respostas HTTP que vamos usar
 - 200 - OK
 - GET
 - Recurso obtido e transmitido no corpo da mensagem
 - PUT ou POST
 - Recurso que descreve o resultado da ação é transmitido no corpo da mensagem
 - 201 - Created
 - POST/PUT
 - Requisição bem sucedida. Recurso criado
 - 400 - Bad Request
 - O servidor não pode ou não vai processar a solicitação devido a algum erro do cliente



Sistema Agenda

- Testando API
 - Baixar o código do github
 - <https://github.com/robertocferreira/sistema-agenda/archive/refs/heads/main.zip>
 - Rodar o projeto fazendo as instalações das dependências
 - Análise rápida do código
 - Fazendo requisições com o postman



Sistema Agenda

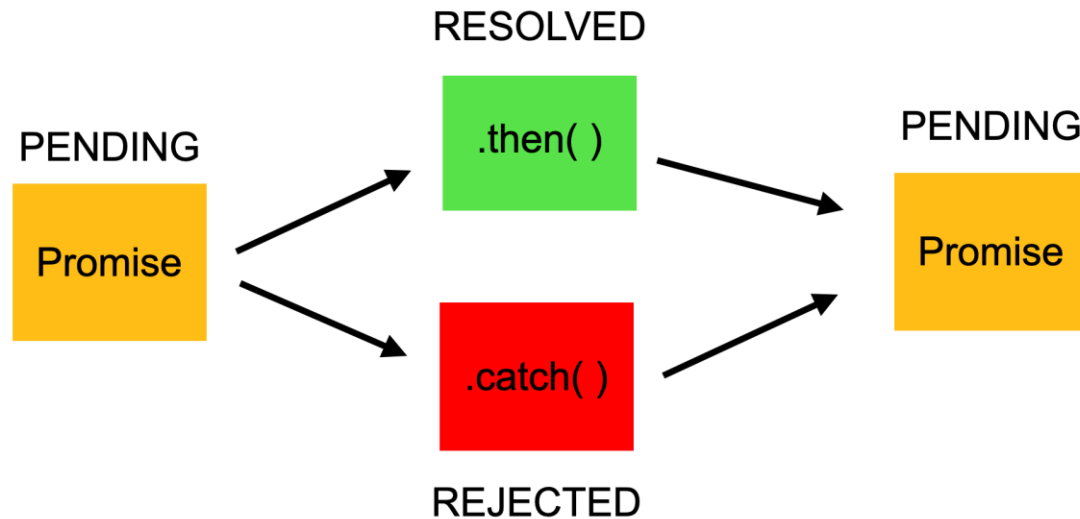
- Fazendo requisições HTTP com o navegador via fetchAPI
 - A API fetch provê ao navegador uma interface para a execução de requisições HTTP através de Promises

```
fetch(input: string,  
  {  
    method?: "GET" | "POST" | "PATCH" | "PUT" | "DELETE",  
    mode?: "navigate" | "same-origin" | "no-cors" | "cors",  
    headers?:  
      {  
        [ key: string ]: any  
      }  
    body?: string  
  }): Promise<Response>
```



Sistema Agenda

- Promise???



Sistema Agenda

```
<script>
  // Função que retorna uma Promise
  function exemploPromise() {
    return new Promise((resolve, reject) => {
      // Simular uma operação assíncrona com setTimeout
      setTimeout(() => {
        const sucesso = true;

        if (sucesso) {
          resolve('A operação foi bem-sucedida!'); // Se tudo correr bem
        } else {
          reject('A operação falhou.');
```



Sistema Agenda

```
// Usando a Promise
exemploPromise()
  .then(result => {
    // Manipula o resultado se a Promise for resolvida com sucesso
    console.log(result);
  })
  .catch(error => {
    // Manipula o erro se a Promise for rejeitada
    console.error(error);
  });
</script>
```



Sistema Agenda

- Async/Await

```
//Função assíncrona que substitui o .then .catch
async function promiseComAsyncAwait(){
  try {
    const resposta = await exemploPromise();
    console.log(resposta);
  } catch (e){
    console.log(e);
  }
}

promiseComAsyncAwait();
```

